pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Overfitting, Dropout, and Early Stopping

WEEK 2 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Overfitting, and three brakes

Course 4's Polish training rows

4,680
companies

326
bankrupt

112 network inputs into a 64-32 network: thousands of weights for 326 bankruptcies.

Three brakes, each measured

L2: a price on large weights

Dropout: a fifth of values off, in training

Early stopping: keep the best epoch

Then all three by five folds

Few rows and many weights: a network can fit its own rows, noise and all.

How overfitting shows

1. Many weights

can learn the fit rows, noise too

2. Fit loss

keeps falling

3. Held-back loss

falls, then turns up

4. The gap

is overfitting

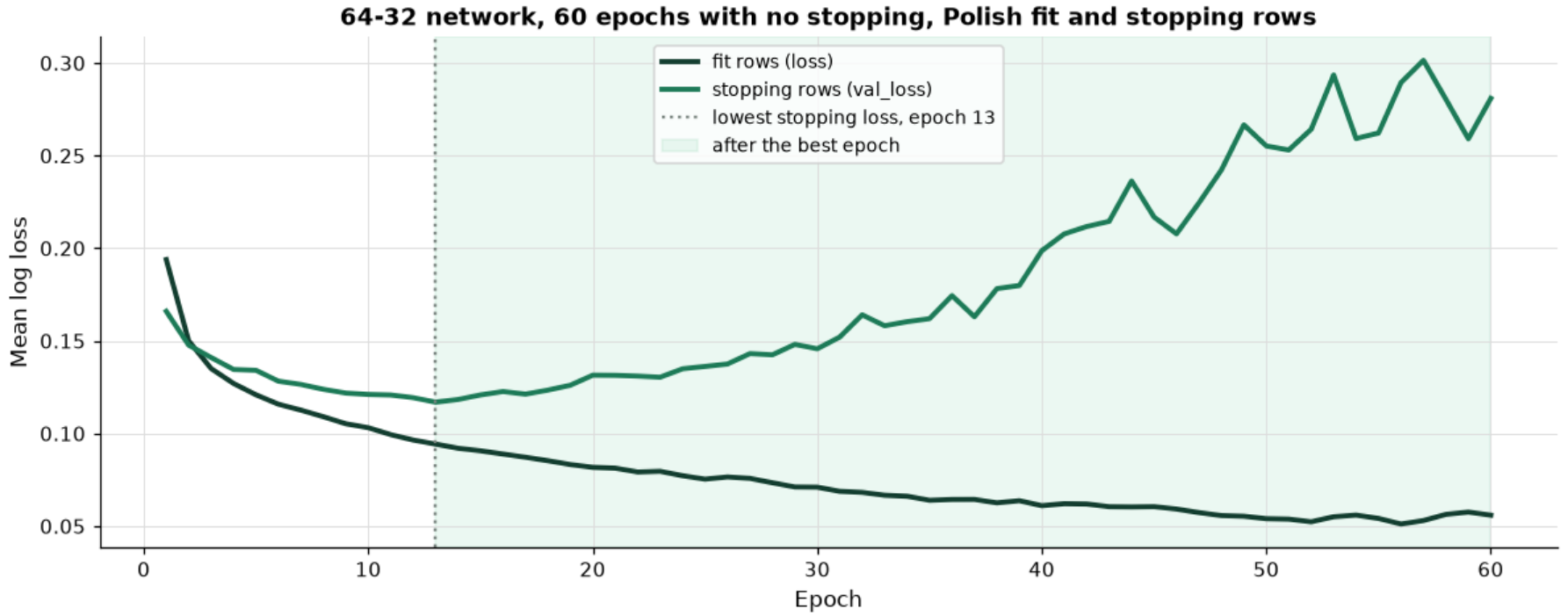
You control

The network's size, the epochs, and three brakes.

What comes out

Two curves, and the epoch where they part.

Overfitting made visible



Stopping loss 0.1169 at epoch 13, 0.2809 at epoch 60; fit loss 0.0559. Average precision after 60: 0.7109.

How L2 works

1. A price

factor x sum of squared weights

2. Added to the loss

so weights grow only if worth it

3. Every step

pulls weights toward 0

4. Biases

are not charged

You control

The factor, and which layers carry it.

What comes out

Smaller weights; a loss with the penalty inside.

L2: a price on large weights

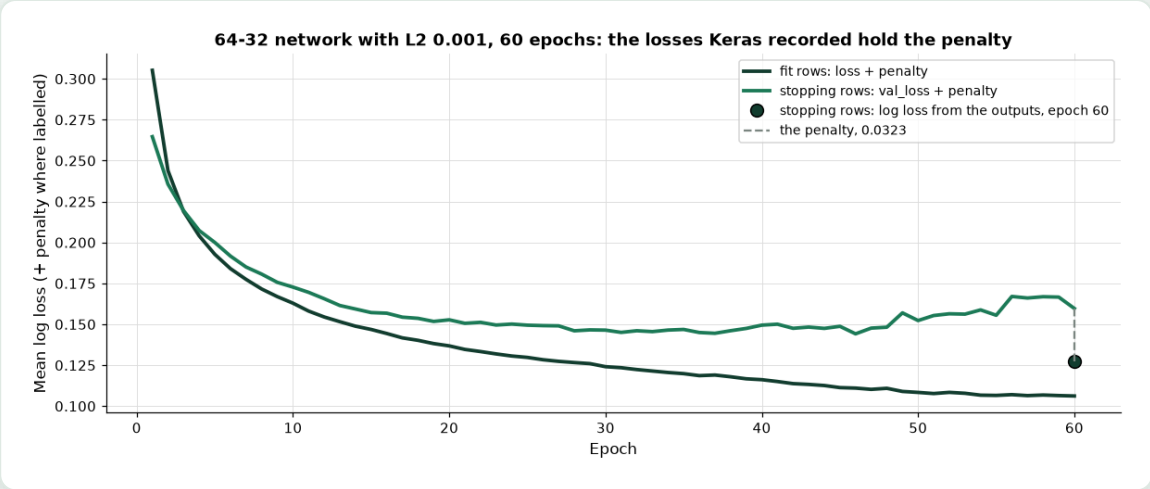
```
loss = mean log loss + 0.001 * (sum of squared hidden kernel weights)
keras.layers.Dense(64, activation="relu", kernel_regularizer=keras.regularizers.L2(0.001))
```

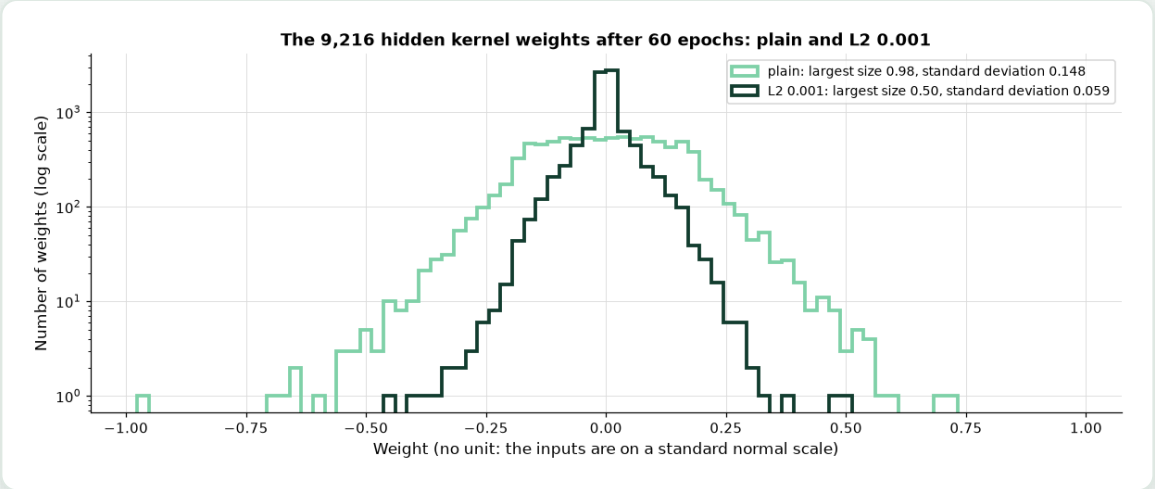
Keras's stopping loss 0.1597 = log loss 0.1274 + penalty 0.0323: the penalty is in every loss it reports.

After 60 epochs	Stopping log loss	Average precision	ROC AUC	Squared weights
plain	0.2604	0.7109	0.9049	203.1065
L2 0.001	0.1274	0.7865	0.9421	32.2599

Compare an L2 network by the log loss of its outputs, never by its val_loss.

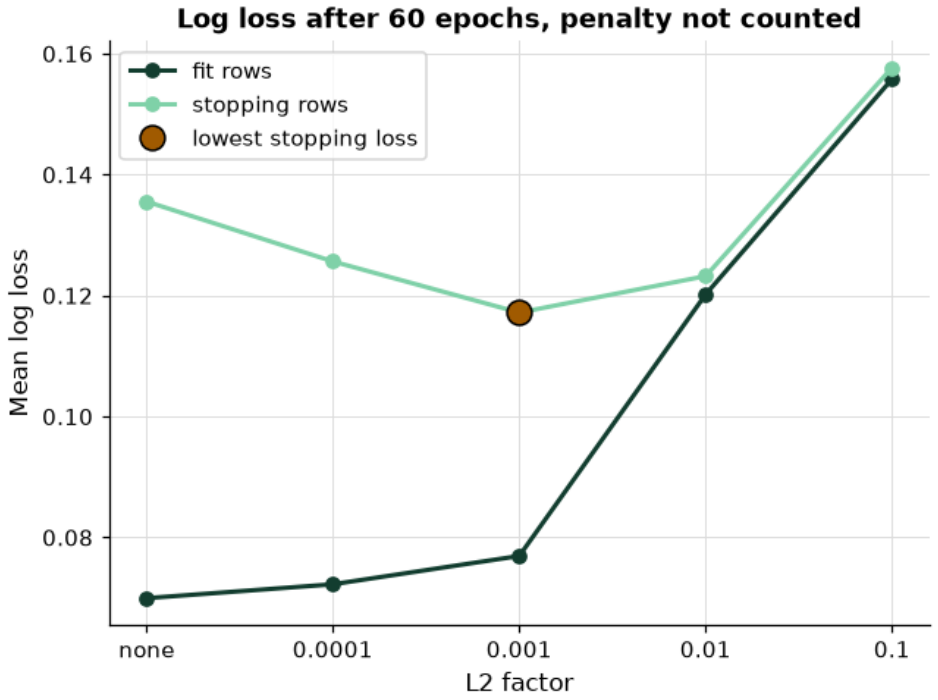
What the penalty does, drawn

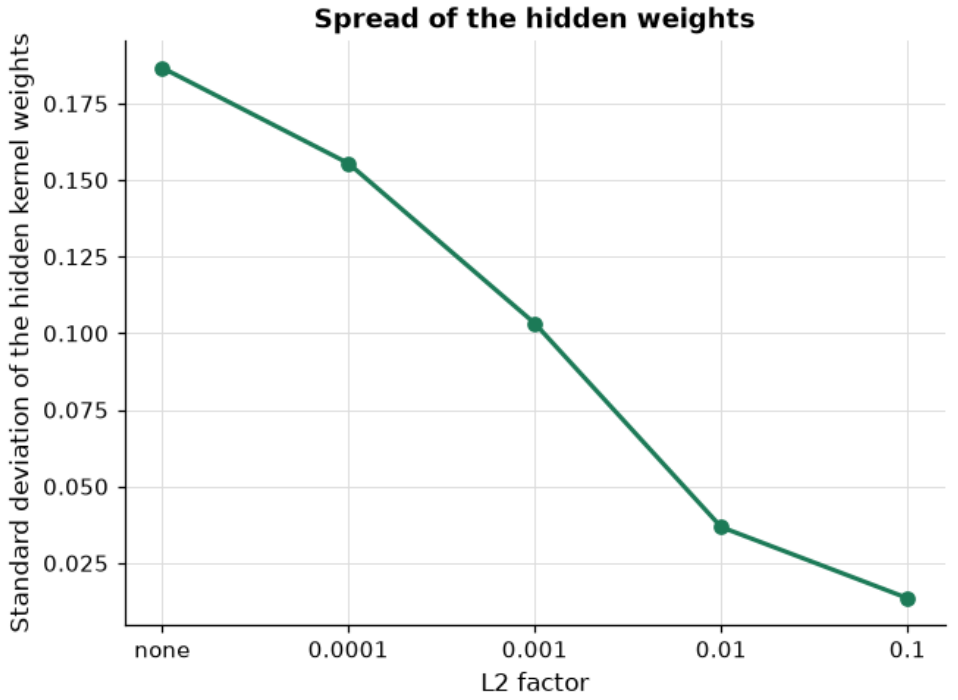




Left: the dashed gap at epoch 60 is the penalty, 0.0323. Right: the plain network's weights spread much further.

Five L2 factors





The stopping loss falls then rises; the dip marks the factor. One run each on one carve: the folds test it.

How dropout works

1. In training

each value zeroed with probability = rate

2. Kept values

divided by 1 - rate

3. At scoring

nothing dropped

4. Units

cannot rely on one another

You control

The rate, and where the layers sit.

What comes out

A network that cannot specialise, with a noisier fit loss.

Dropout by hand

```
dropped = values * (uniforms >= 0.2) / (1 - 0.2)        # netkit.dropout_by_hand
```

20 values, seeded uniforms

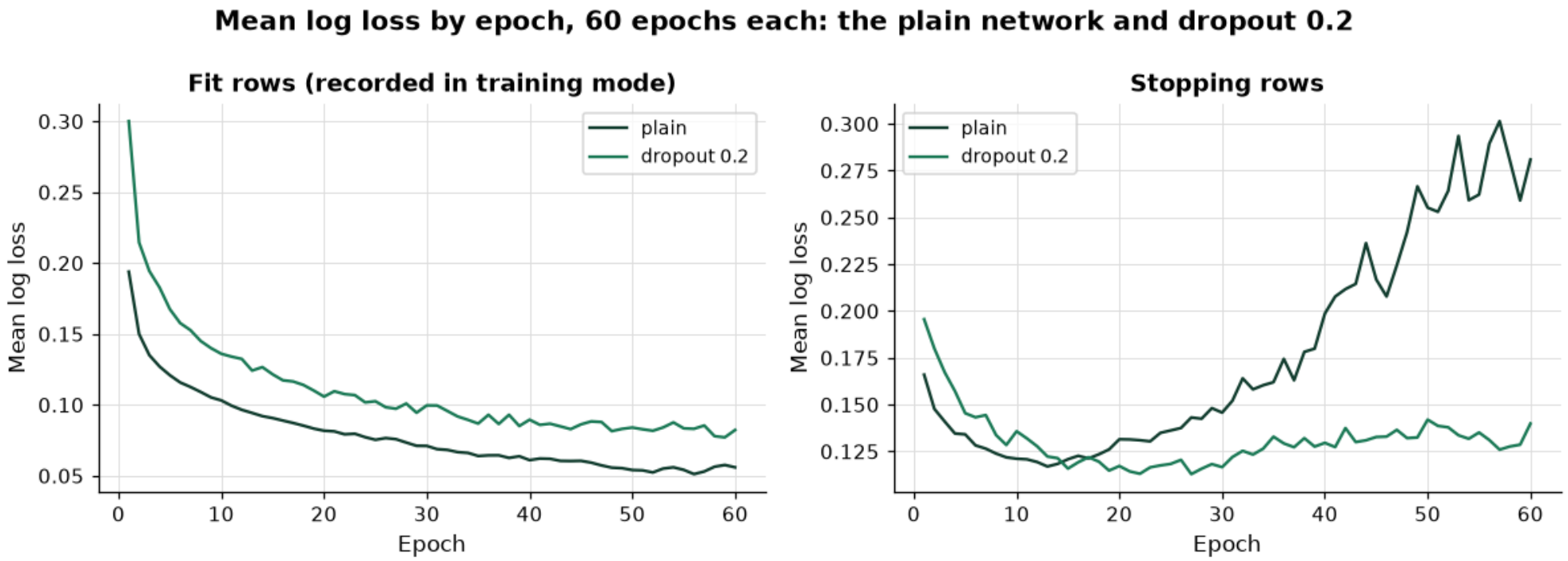
3 values whose draw fell below 0.2 become 0; the rest are multiplied by 1.25, so each value keeps its average.

Training against scoring

In fit: a fresh fifth dropped at every batch. In predict: nothing dropped, nothing divided; the layer passes values through.

Dropout happens while a network is fitted, never while it is scored.

Dropout against no brake



After 60 epochs: stopping log loss 0.1339 against 0.2604; average precision 0.7829 against 0.7109.

The paper that named the layer

Srivastava, Hinton, Krizhevsky, Sutskever, and Salakhutdinov, 2014

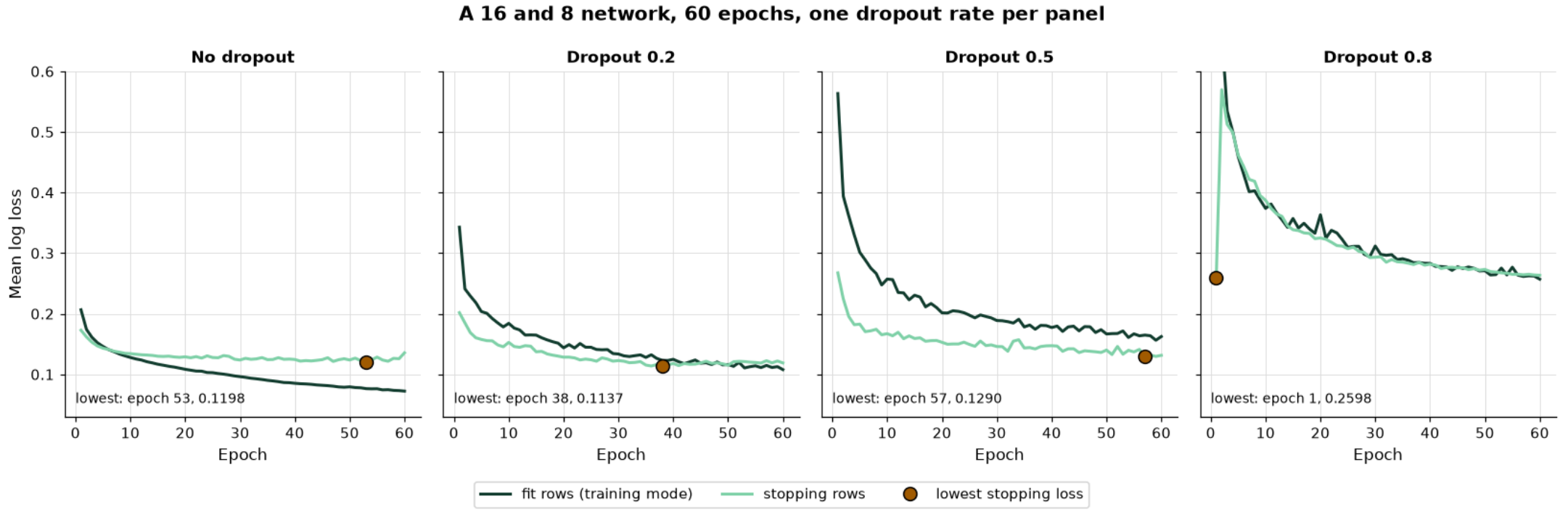
"Dropout: A Simple Way to Prevent Neural Networks from Overfitting"

Journal of Machine Learning Research, volume 15, number 56, pages 1929 to 1958

Its title holds both of today's words: the layer, and the problem it is for.

Source, read 6 October 2026: the paper's page at the journal, jmlr.org/papers/v15/srivastava14a.html. Bibliographic facts only; the paper itself was not read for this course.

Four dropout rates



Pick the rate whose stopping curve is lowest and stays close to the fit curve. Too high a rate underfits.

How early stopping works

1. A callback

fit consults it every epoch

2. Watch

the stopping rows' loss

3. Wait

patience epochs, then stop

4. Restore

the best epoch's weights

You control

What is watched, patience, min_delta, restore_best_weights.

What comes out

A fit that ends on its own, and a kept epoch.

Early stopping, with the best weights kept

```
stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True)
model.fit(A_fit, y_fit, validation_data=(A_stop, y_stop), epochs=200, callbacks=[stop], verbose=0)
```

23

epochs run of 200

13

best epoch, weights kept

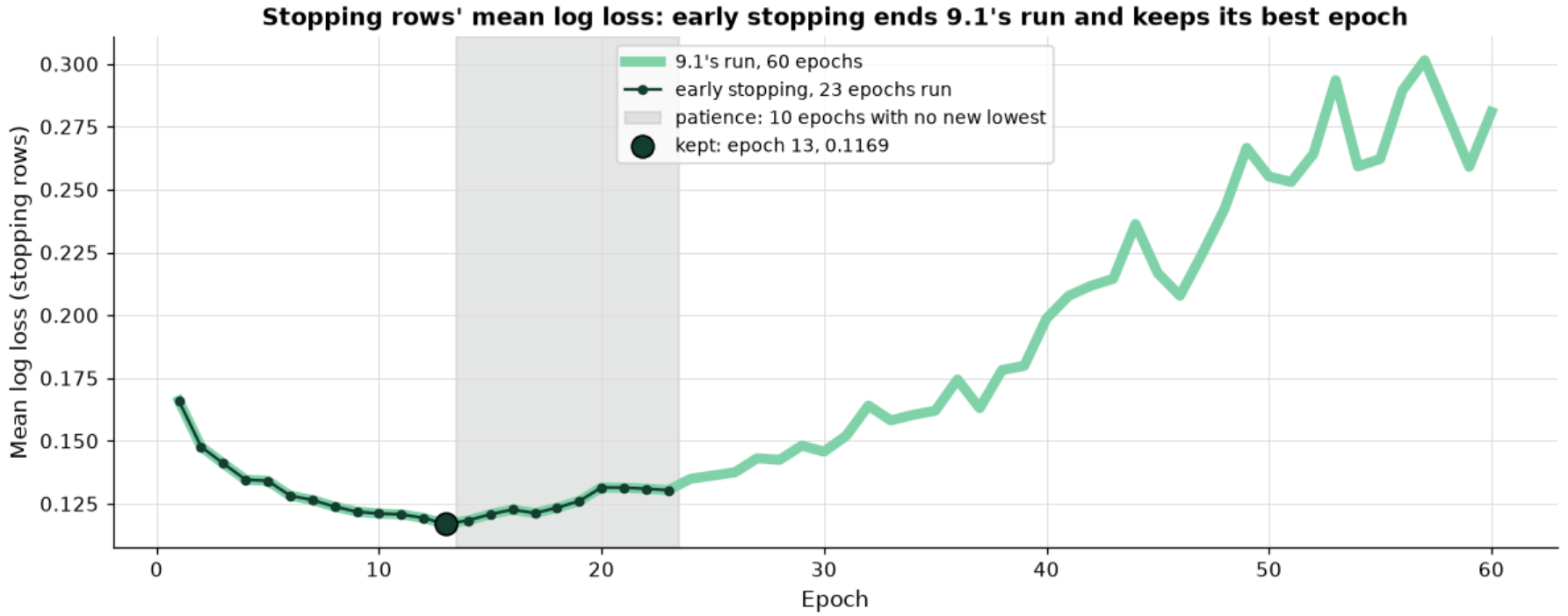
0.7535

average precision (60 epochs: 0.7109)

The stopping rows chose the epoch, so their score flatters it a little: section 9.5 scores by folds.

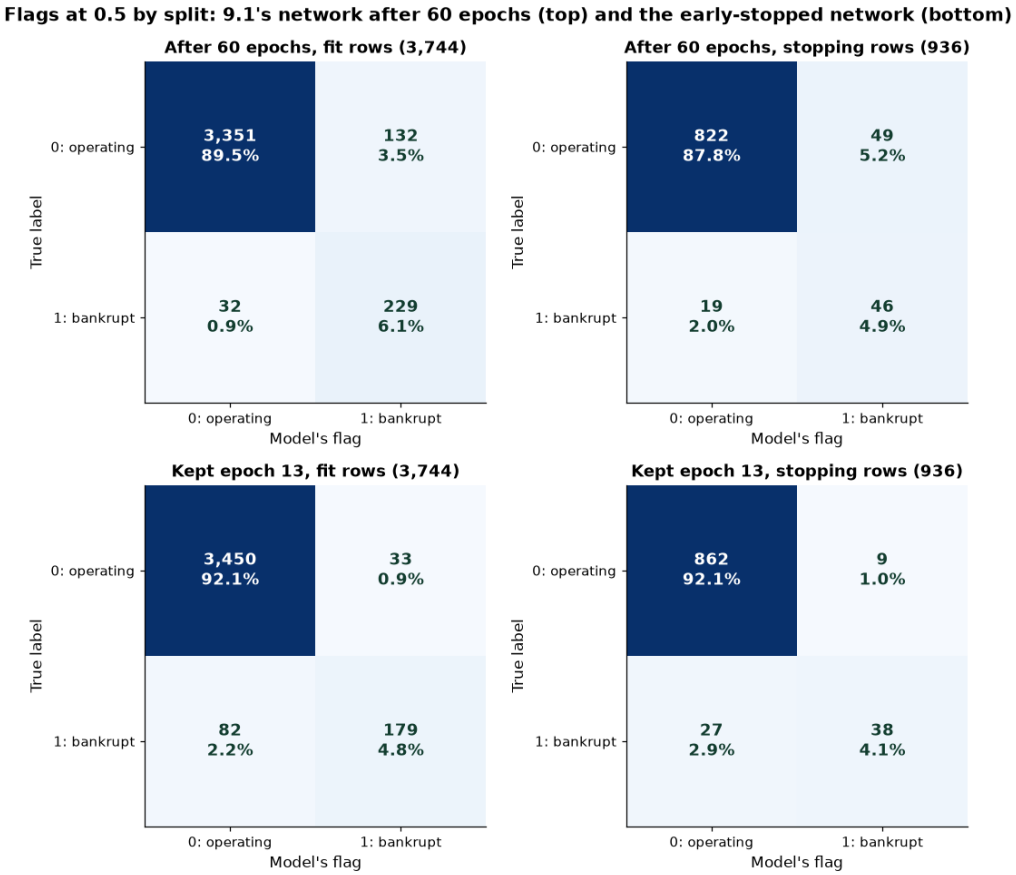
Same seed, same rows: the callback kept the epoch the chart marked.

The callback, on the curve



Epochs 14 to 23 are the patience; epoch 13's weights came back.

Flags by split, at 0.5



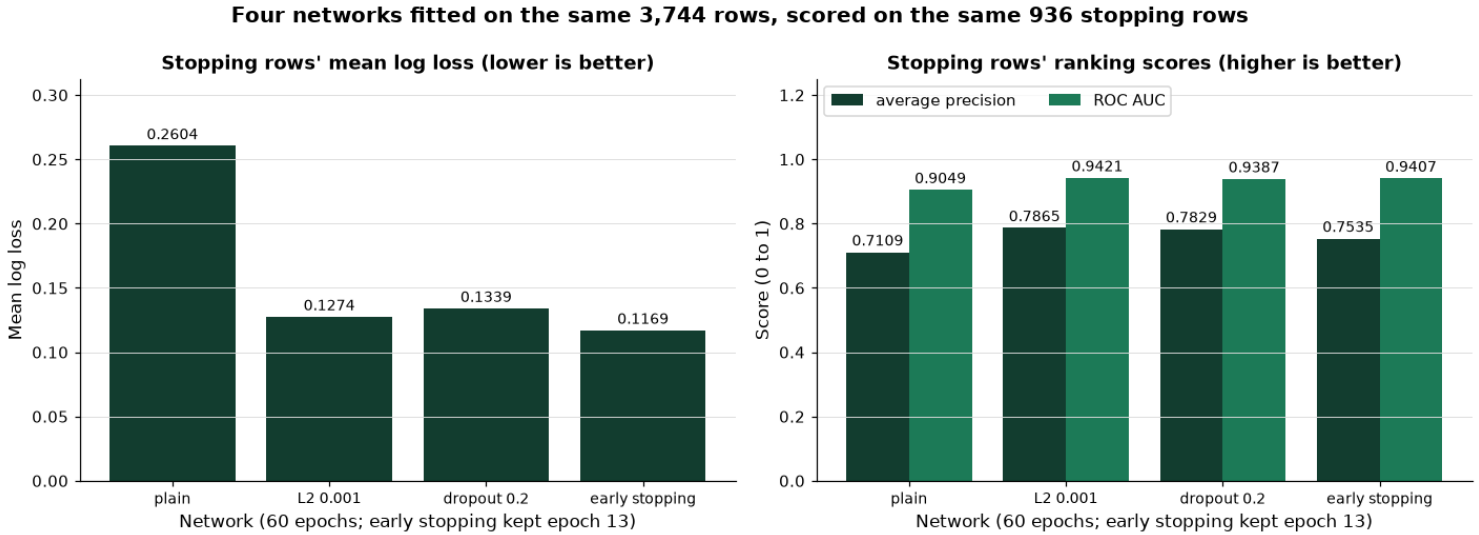
How to read it

What happened down the side, the network's flag across the top; each cell its count and its share of the split.

Compare left with right

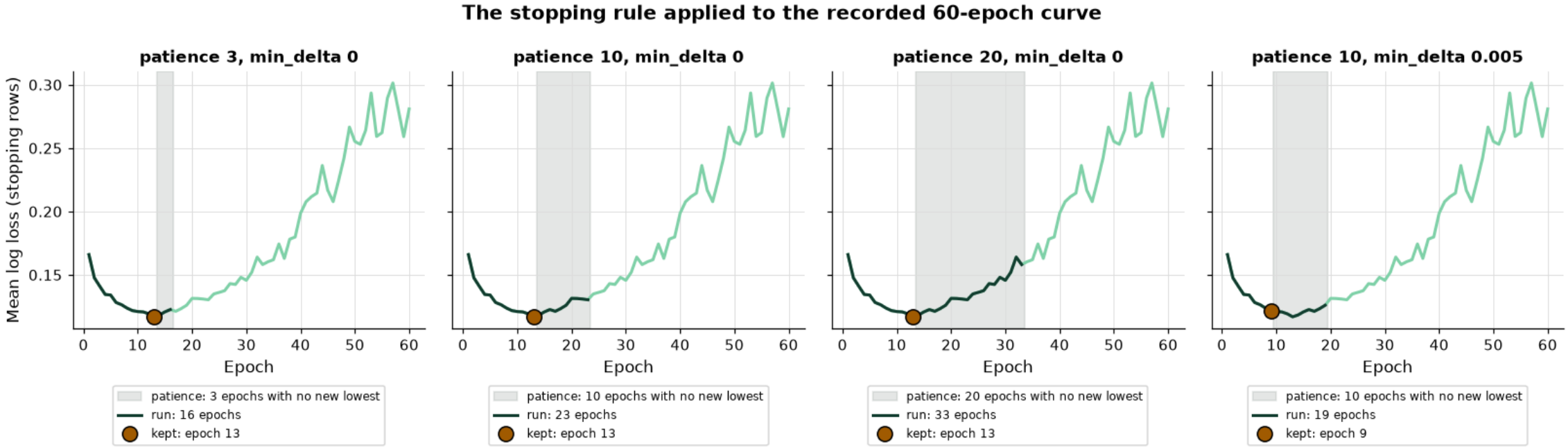
The network left to overfit finds far more bankrupt companies in its own rows than in the stopping rows; the kept network's gap is smaller.

Four networks, one carve



Network	Stopping log loss	Average precision	ROC AUC
plain	0.2604	0.7109	0.9049
L2 0.001	0.1274	0.7865	0.9421
dropout 0.2	0.1339	0.7829	0.9387
early stopping	0.1169	0.7535	0.9407

The stopping rule, four settings



Patience sets how many epochs are spent; min_delta can move the kept epoch earlier than the lowest.

Every setting of the three brakes

Setting	Default	What it controls	Tune first
L2: l2	0.01	price on every squared weight	tune first
Dropout: rate	required	share of values zeroed in training	tune first
EarlyStopping: min_delta	0	smallest fall that counts	tune first
EarlyStopping: patience	0	epochs with no improvement before it stops	tune first
EarlyStopping: restore_best_weights	False	put back the best epoch's weights	tune first

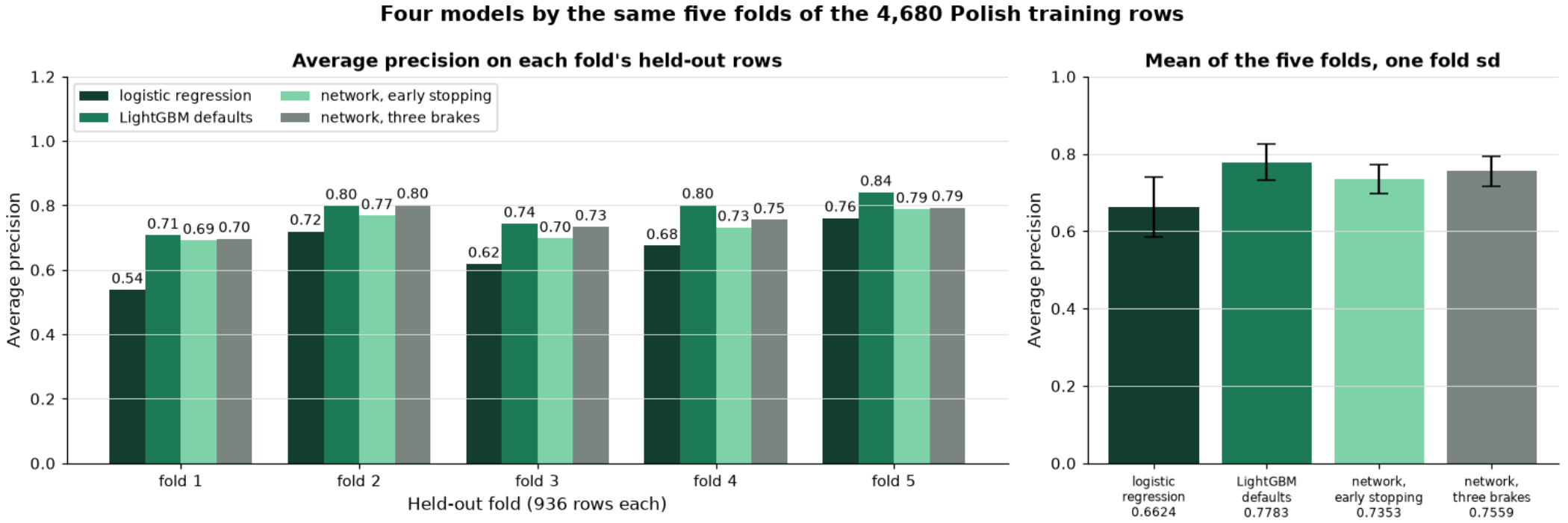
12 settings in all; the notebook's table lists every one. The course overrides three defaults: patience, restore_best_weights, and the L2 factor.

The four together, by five folds

Model, five folds inside 4,680 rows	Average precision	Fold sd	ROC AUC
Course 4 logistic regression	0.6624	0.0780	0.9281
Network, early stopping	0.7353	0.0380	0.9375
Network, L2 + dropout + stopping	0.7559	0.0387	0.9425
LightGBM defaults	0.7783	0.0469	0.9509

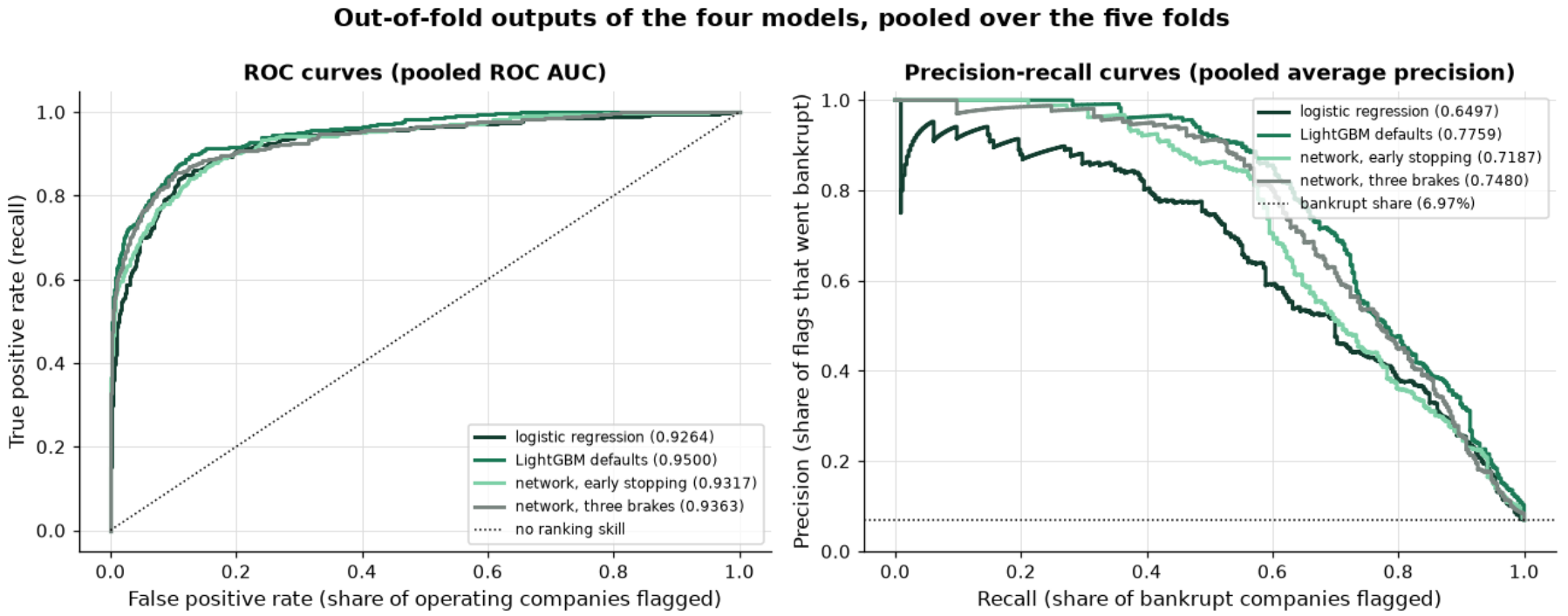
The brakes gain less than a fold sd; LightGBM stays ahead by less than a fold sd. Day 10 adds intervals.

Fold by fold



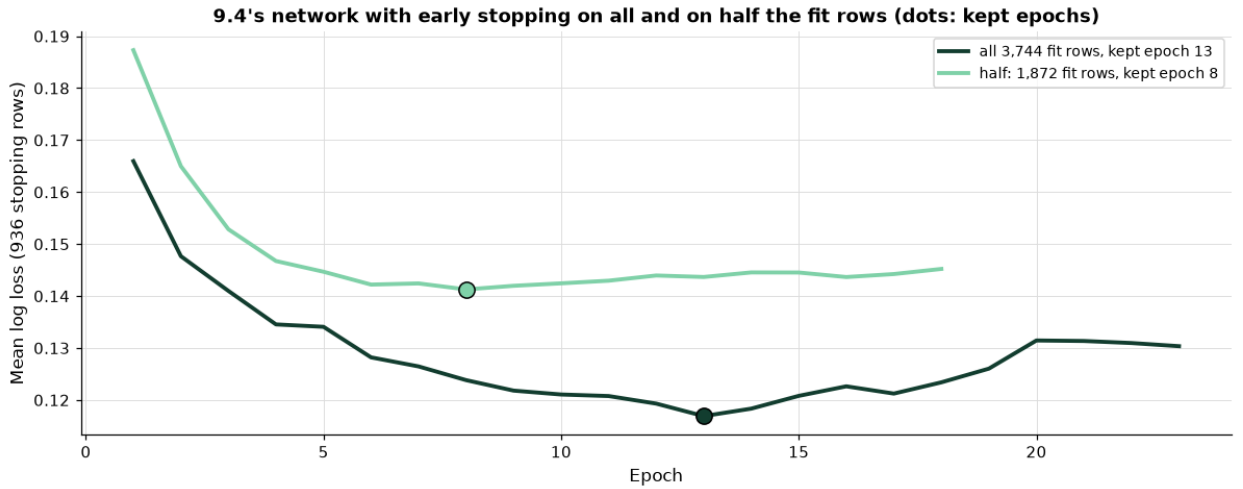
The folds move every model together; the spread across folds dwarfs the gaps between models.

ROC and precision-recall, pooled



The ROC curves bunch together; precision-recall spreads them out, because bankrupt companies are few.

How many rows a network needs



Fit rows	Bankrupt	Inputs	Best epoch	Lowest stopping loss	Average precision
3,744	261	112	13	0.1169	0.7535
1,872	131	95	8	0.1413	0.6777

What the network is not

Allowed names

model probability of bankruptcy in this file

flagged at threshold t

the model's score (class-weighted)

Never called

a credit view on any company

a rating

an investment decision

Allowed names

weight

gradient

the network leans on

Never called

driver or key driver

cause or reason

what the network knows

A brake changes how the network fits its rows. It says nothing about why a company failed.

Excel side by side

Excel side by side (tested in Excel)

`=A2*(B2>=0.2)/(1-0.2)`	filled down: dropout_by_hand within 1e-12
`=COUNTIF(C2:C21,0)`	3 values dropped
`=0.001*SUMSQ(A1:BL112,BN1:CS64)`	0.032260, l2_penalty's number
`=MATCH(MIN(B2:B61),B2:B61,0)`	epoch 13, best_epoch's number

No network training and no early stopping in Excel. Each brake's arithmetic is a formula.

Save the day: a change inside a line

```
git diff --word-diff --word-diff-regex="[^,]+" results.csv
```

```
network_l2_dropout_early_stopping,cv,0.9425,0.7559,[-0.5000,0.9626,0.8578,0.5552,0.6741-]
    {+0.2006,0.9489,0.6173,0.7025,0.6571+}
```

[-old-] and {+new+} inside the line: only the threshold and the four flag columns change. ROC AUC and average precision stay, because a threshold does not reorder the companies.

A word is anything between commas: in a CSV, a cell.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: scored with training=True: average precision 0.7530, then 0.7106, against predict's 0.7901.

Code that runs is not the same as code that is right.

What you can do now

- Read loss curves and say where overfitting starts
- Price large weights with L2, and allow for the penalty
- Do inverted dropout by hand; score with predict
- Stop early and keep the best epoch's weights
- Compare brakes by five folds beside LightGBM

Week 2, Day 5 Case study: class weights on bankruptcy